Algoritmo Genético Distribuído em Modelo de Ilhas para o Problema do Caixeiro Viajante

[Nome do Aluno 1], [Nome do Aluno 2], [Nome do Aluno 3],
[Nome do Aluno 4], [Nome do Aluno 5]

[Curso/Departamento] — [Instituição]

{aluno1,...,aluno5}@[dominio]

Resumo. Este trabalho descreve um sistema distribuído que executa um algoritmo genético (AG) em modelo de ilhas para o Problema do Caixeiro Viajante. Cada nó mantém uma subpopulação e troca indivíduos com o vizinho em anel por RPC. O sistema implementa manualmente relógios lógicos de Lamport e eleição de líder pelo algoritmo Bully, além de tolerar falhas de nós. Em testes locais com 80 cidades, quatro ilhas com migração obtiveram tour médio 3% melhor que uma ilha isolada de mesmo tamanho, e o sistema concluiu a execução mesmo após a queda do líder.

1. Problema escolhido

Escolhemos o tema *Algoritmo Genético Distribuído*. O problema computacional é o Problema do Caixeiro Viajante (TSP): dado um conjunto de n cidades, encontrar o ciclo de menor comprimento que visita todas uma única vez. Trata-se de um problema NP-difícil, e o AG o aborda evoluindo uma população de permutações. A avaliação de aptidão e a reprodução são independentes entre indivíduos, o que permite dividir o trabalho em unidades executadas em máquinas distintas.

Adotamos o *modelo de ilhas*: a população total é particionada em subpopulações, uma por nó, que evoluem de forma independente e, a cada 25 gerações, enviam seus 4 melhores indivíduos ao vizinho no anel. A migração injeta diversidade nas ilhas e propaga boas soluções sem sincronização global. A instância (80 cidades em $[0, 1000]^2$) é gerada por uma semente fixa, de modo que todos os nós conhecem o problema sem transferi-lo pela rede.

2. Arquitetura do sistema

Todos os nós executam o mesmo programa e possuem três componentes: (i) um servidor XML-RPC multithread; (ii) uma thread de evolução (a ilha do AG e a migração); e (iii) uma thread monitora (*heartbeat*, eleição e, no líder, agregação). Há um único papel especial, o **coordenador** (líder), eleito dinamicamente: ele envia *heartbeats*, agrega os melhores resultados de cada ilha, detecta o término e grava o resultado final. A Figura 1 mostra quem se comunica com quem, e a Tabela 1 lista as mensagens.

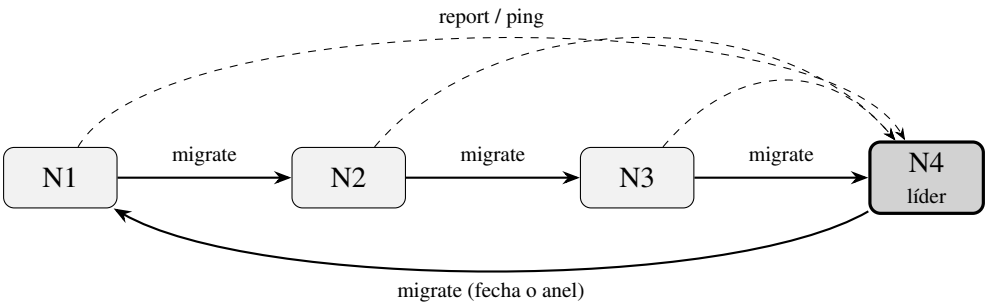


Figura 1. Topologia com 4 nós. Linhas cheias: migração em anel. Tracejadas: `report` e `ping` entre seguidores e líder. `election` e `coordinator` podem ser trocadas entre quaisquer nós.

3. Implementação dos requisitos

Atendemos três requisitos da lista do enunciado, todos implementados manualmente sobre o RPC.

Tabela 1. Mensagens RPC. Toda chamada leva (remetente, timestamp de Lamport) e a resposta devolve o timestamp do respondente.

Método	Trajeto	Conteúdo principal
`ping`	líder ↔ seguidores	vazio; resposta traz o líder conhecido
`election`	nó → IDs maiores	vazio; resposta `OK`
`coordinator`	novo líder → todos	anúncio; `REJECT` se receptor é maior
`migrate`	nó → vizinho no anel	*k* melhores tours (listas de cidades)
`report`	nó → líder	geração, melhor tour/custo, `done`
`shutdown`	líder → todos	encerra o cluster

3.1. RPC

Usamos o XML-RPC da biblioteca padrão do Python (o enunciado permite qualquer RPC; gRPC não estava disponível no ambiente de desenvolvimento e XML-RPC dispensa dependências). Cada nó expõe seis métodos. Do lado cliente, `Node.call` cria um proxy com *timeout* (1 s), captura qualquer erro de rede e devolve `(ok, resultado)`, de modo que falhas de comunicação nunca derrubam o nó. Cada chamada incrementa em 1 o contador de falhas consecutivas do destino; três falhas o marcam como morto, e um sucesso zera o contador. Como todo RPC (migração, relatório, *ping*) alimenta esse contador, a detecção ocorre rapidamente. O servidor atende cada requisição em uma thread própria.

3.2. Relógios lógicos de Lamport

Cada nó mantém um `LamportClock` protegido por *lock*. Antes de enviar uma mensagem, o remetente executa $L \leftarrow L+1$ e anexa L; ao receber, o destinatário executa $L \leftarrow \max(L,t)+1$, e o mesmo vale para a resposta. Todos os eventos relevantes são registrados em log com o valor de L. Exemplo real: N1 enviou `migrate` com $t = 5$; N2, cujo relógio estava em 11, passou para $\max(11,5)+1 = 12$. Os logs permitem verificar a relação *aconteceu-antes* (um envio sempre tem L menor que o recebimento correspondente). O relógio é usado para ordenar causalmente os eventos de eleição e migração na depuração, já que os relógios físicos das máquinas não são sincronizados.

3.3. Eleição de líder (Bully)

Cada nó possui um ID inteiro; vence o maior ID vivo. Um nó inicia a eleição ao subir ou ao detectar a queda do líder. Ele envia `election` a todos os IDs maiores. Se nenhum responde `OK`, ele se torna líder e envia `coordinator` a todos. Se algum responde, aguarda o anúncio por até 3 s e, se este não chegar, repete a eleição. Quem recebe `election` de um ID menor responde `OK` e inicia a própria eleição. Um `coordinator` vindo de ID menor é rejeitado, o que resolve a situação de dois líderes (por exemplo, quando um nó de ID alto volta). Seguidores fazem *ping* ao líder a cada 1 s; o líder faz *ping* a todos. Ao assumir, o novo líder zera a agregação, e os nós que já haviam terminado reenviam seu `report` final.

3.4. Tolerância a falhas e término

Se o vizinho do anel não responde, a migração tenta o próximo do anel. O líder só finaliza quando todos os nós *vivos* informaram `done`; então grava o resultado (JSON) e envia `shutdown`.

4. Resultados experimentais

Os testes foram feitos em *localhost*, com processos independentes, população total 240, 1500 gerações, 80 cidades e 3 sementes por configuração. A referência da heurística do vizinho mais próximo é 8406,7 (menor é melhor). A Tabela 2 resume os resultados.

A maior parte da diferença entre o caso de 1 nó (população 240) e os demais se deve ao tamanho da subpopulação: com número fixo de gerações, populações menores convergem mais

Tabela 2. Comprimento do melhor tour (média e desvio padrão de 3 execuções).

Configuração	Média	Desvio
1 nó, população 240	9938,4	297,6
1 nó isolado, população 60 (controle)	7478,7	105,5
2 nós × 120	7350,3	67,1
4 nós × 60	7262,3	111,2
5 nós × 48	7295,6	158,9

rápido, como mostra o controle de 1 nó com população 60. O efeito da migração isolado é modesto: 4 ilhas ficaram cerca de 3% melhores que uma ilha isolada de mesmo tamanho. Com 5 nós o resultado não melhorou, indicando que ilhas pequenas demais perdem diversidade. Com três sementes, as diferenças entre 2, 4 e 5 nós estão dentro do desvio e não são conclusivas.

Falhas injetadas (4 nós, 3000 gerações): (i) ao matar o líder N4 aos ∼4 s, N3 detectou a falha em menos de 1 s, venceu a eleição e a execução terminou com tour 7003,9; (ii) ao matar o nó N2, N1 passou a migrar para N3 e a execução terminou com tour 7343,8. A população de um nó morto é perdida, pois não há *checkpoint*.

Limitação de medição: o ambiente de teste tinha um único núcleo de CPU, portanto *não medimos speedup*; o `benchmark.py` deve ser executado em máquinas distintas da rede local para isso.

5. Desafios e problemas encontrados

- **Confundir efeito da distribuição com efeito do AG.** A comparação ingênua entre 1 nó e N nós superestimava o ganho; foi preciso um experimento de controle com mesma subpopulação.
- **Concorrência.** Chamadas RPC chegam em threads diferentes da evolução; protegemos o relógio, a caixa de imigrantes e o estado de agregação com *locks*, e evitamos eleições simultâneas com uma *flag* sob *lock*.
- **Detecção de falhas e tempestade de eleições.** Na partida, todos os nós iniciam eleição ao mesmo tempo; a *flag* de eleição em andamento e o `REJECT` evitam múltiplos líderes.
- **Término com falhas.** O líder precisa saber quem ainda está vivo para não esperar eternamente; isso exigiu o contador de falhas e o reenvio do `report` final após troca de líder.
- **Ambiente.** gRPC não estava disponível sem rede, o que motivou o XML-RPC; os *timeouts* exigiram um *transport* próprio.

6. Papel de cada aluno

- **Aluno 1:** arquitetura, protocolo de mensagens e camada RPC (`rpc.py`, `Node.call`), *timeouts* e tratamento de falhas de comunicação.
- **Aluno 2:** eleição Bully, *heartbeat* e detecção de falha do líder.
- **Aluno 3:** relógio de Lamport, integração nas mensagens e formato dos logs.
- **Aluno 4:** algoritmo genético (TSP, OX, mutação, seleção), ilhas e política de migração.
- **Aluno 5:** `run_local.py`, `benchmark.py`, injeção de falhas, experimentos, README e relatório.

O código está disponível no projeto entregue, com instruções no `README.md`.